Course Notes - Arrays and Linked Lists
An array stores elements in contiguous memory.
Index access is O(1): the address of element i is base + i * size.

A linked list stores nodes with a value and a pointer to the next node.
Insertion at the head is O(1), but there is no random access - reaching element i costs O(i).

Choose arrays when you read by index often; choose linked lists when you insert or delete at known positions frequently.

Course Notes - Stacks, Queues and Hash Maps
A stack is LIFO: push adds to the top, pop removes from the top.
Typical uses: undo history, expression evaluation, DFS.

A queue is FIFO: enqueue adds at the rear, dequeue removes from the front.
Typical uses: task scheduling, BFS, buffering.

A hash map hashes each key into a bucket. Average lookup, insert and
delete are O(1); worst case is O(n) on collisions. A good hash function
and a sensible load factor keep buckets short.

Course Notes - Trees, Heaps, Graphs and DP

A binary search tree keeps keys sorted; a balanced BST (height ~ log n) supports search, insert and delete in O(log n).

A heap is a complete tree satisfying the heap property. It powers priority queues: insert and extract-min are O(log n), peek is O(1).

Graphs are vertices and edges. BFS explores level by level (shortest path in unweighted graphs); DFS explores deep first (cycle detection, topo sort).

Dynamic programming caches subproblem results when a problem has optimal substructure and overlapping subproblems - turning exponential into polynomial time.